

NimbusPay ticket triage

Selected run C | Dev exact match 62.5% | Mean field accuracy 93.9%

Fresh default-notebook reproduction: Run C trained on a new free T4 in 22.14 minutes; the complete notebook took 79.52 minutes, including setup, evaluation, 400 test predictions and adapter reload. Rerun dev exact match was 61.0%, mean field accuracy 93.75%; all 10 seeded reload predictions matched. This verifies the under-three-hour pipeline; the five-run comparison above remains the basis for selecting the submitted adapter. The rerun score is reported separately because the fresh training produced a different checkpoint.

Training-only audit and evidence

Parsed all 1,445 training rows and validated JSON keys, field types, schema invariants and source evidence. Retained 1,333; fixed 194 original rows and dropped 112. Finding counts overlap across fixes. The original archive is preserved privately.

Dropped 25 incomplete labels, 10 empty/test tickets, and 77 duplicate tickets (60 byte-identical plus 17 whitespace-equivalent). Kept the earliest ID after corrected-label agreement. Converted 70 legacy labels. Ticket/schema checks repaired 25 amounts with Decimal, 30 dates, 20 IDs, 60 priorities and 21 human-escalation fields.

Quoted-history lines are excluded only from correction evidence, while full input text is retained. Amount distractors, scratch-card mentions and prior ticket numbers are checked explicitly. Category/language labels for substantive tickets are retained; this is a targeted audit, not exhaustive relabelling. Every changed original ID appears once in `cleaning_log.csv`. Private evidence records before/after values and source text; nine audit tests and repeat-run byte comparisons verify determinism.

Measured baselines and controlled comparisons

Run / change	Exact	Fields	Peak GiB A/R	Train min
Base, fixed prompt (FP16)	0.0%	0.0%	-	-
Base + schema (FP16)	0.0%	0.0%	-	-
A: raw labels, 4-bit	58.5%	92.9%	6.38/7.54	31.5
B: cleaned, 4-bit	61.5%	93.4%	6.31/7.54	31.0
C: B with FP16 LoRA	62.5%	93.9%	7.06/8.22	22.9
D: B with LR 1e-4	54.5%	92.1%	6.31/7.54	32.1
E: B with notation variants	62.5%	93.6%	6.31/7.54	31.7

A/R = peak CUDA allocated/reserved during training; elapsed excludes model load and dev generation. A vs B isolates cleaning, B vs C precision, B vs D learning rate, and B vs E input notation. All use one seed and 160 updates; cleaning changes effective epochs. Treat differences as exploratory, without statistical confidence intervals.

The fixed baseline wraps all 200 responses in Markdown fences, giving 0% under the unchanged strict scorer. Its first output also uses a different ticket schema. JSON validity in the table means parseable JSON object, not complete schema compliance.

Pipeline and selection

Qwen2.5-1.5B-Instruct (1.54B), pinned revision 989aa7980e4c, on Tesla T4. PEFT trains 18,464,768 parameters. Rank 16, alpha 32, dropout .05 on attention/MLP projections; AdamW, batch 1 x accumulation 16, cosine LR 2e-4 (D: 1e-4), eight warmup steps, FP16 compute and gradient checkpointing. QLoRA uses NF4/double quantization.

Native chat template; prompt/header/padding/separator tokens masked, assistant JSON plus end token supervised. Across raw, clean and augmented sequences, p50/p95/p99/max: 168/225/1816/2215 tokens; bound 2304, dynamic padding, no truncation/packing. All 40 original long rows were measured; 36 unique long tickets remain after deduplication. E replaces 162 eligible tickets (20% of eligible) with deterministic equivalent amount/ID notation, preserving labels and size.

Error analysis: selected adapter

All 200 dev rows, supplied scorer unchanged. JSON valid 100.0%; 75 rows fail exact match. The explanations below are hypotheses grounded in the observed field differences and ticket evidence, not causal proof.

Field (weakest first)	Accuracy	Category	n	Exact	Fields
priority	64.5%	fraud	15	40.0%	89.2%
needs_human	92.5%	refund	55	41.8%	90.7%
category	96.5%	payment_failed	53	43.4%	91.0%
txn_date	98.5%	other	16	87.5%	97.7%
channel	99.5%	offers	18	94.4%	99.3%
amount_paise	100.0%	account_access	19	94.7%	99.3%
txn_id	100.0%	kyc	24	100.0%	100.0%
language	100.0%				

Actual dev failures 1-5

1. dv-00019 (refund) | prediction -> gold

category: "product" -> "refund"; priority: "P2" -> "P1"; needs_human: false -> true

The gold label maps the faulty-product complaint to refund, although a refund is not explicit in the wording. The model emits the disallowed category product and also misses the P1 amount threshold and its human-escalation consequence. Similar ambiguous training complaints need manual review.

2. dv-00046 (fraud) | prediction -> gold

category: "refund" -> "fraud"; priority: "P3" -> "P1"

The statement complaint is labelled fraud in gold; the model chooses refund. That category error also misses the fraud-driven P1 rule. The repeated-contact phrase is handled correctly for needs_human.

3. dv-00181 (fraud) | prediction -> gold

channel: null -> "card"

The bank-card freeze request mentions a strange withdrawal. Fraud and the date are correct, but the model returns a null channel where gold uses card, showing a missed bank-card context association.

4. dv-00090 (payment_failed) | prediction -> gold

txn_date: "2026-07-12" -> "2026-07-10"

Parso means two days before the Received date in this past-tense Hinglish ticket: July 10. The model instead copies July 12, the receipt date, while the other transaction fields are correct.

5. dv-00120 (refund) | prediction -> gold

txn_date: "2026-06-26" -> "2026-06-25"

Parso against June 27 resolves to June 25. The model produces June 26, a one-day rather than two-day offset. This is a relative-date error despite correct amount and transaction-ID normalization.

Failures, next steps and reproducibility

6. dv-00011 (refund) | prediction -> gold

priority: "P3" -> "P1"; needs_human: false -> true

The correctly extracted disputed amount is Rs 84,137, above the Rs 50,000 P1 threshold. The model returns P3 and false for needs_human, missing both the threshold and the dependent escalation rule.

7. dv-00003 (refund) | prediction -> gold

priority: "P3" -> "P1"

The ticket explicitly threatens consumer court, which requires P1 regardless of the small amount. The model returns P3 while needs_human is true, showing inconsistent application of the priority and escalation rules.

8. dv-00005 (payment_failed) | prediction -> gold

priority: "P3" -> "P2"

A failed transfer of Rs 49,680 lies above the Rs 5,000 P2 threshold and below P1. The model extracts the correct amount but assigns P3, so the failure is threshold application rather than amount conversion.

9. dv-00016 (payment_failed) | prediction -> gold

priority: "P3" -> "P2"

The failed UPI top-up of Rs 25,647 requires P2. The model returns P3 despite a correct category and amount; the urgent wording does not change the specified threshold rule.

10. dv-00017 (refund) | prediction -> gold

priority: "P3" -> "P1"

The model correctly ignores the Rs 712 balance and extracts the Rs 180,827.25 refund amount, but returns P3 instead of P1. Its true needs_human value is inconsistent with the predicted priority.

One more week

Days 1-2: manually audit remaining ambiguous training categories and language labels; expand training-only coverage around measured weak rules, threshold boundaries, quoted history, relative dates and competing amounts. Days 3-4: compare balanced sampling and focused amount/date notation augmentation under the same generation contract. Days 5-6: repeat the strongest configuration across three seeds and test a small rank/step budget. Day 7: lock the selected configuration, validate a fresh T4 run and independently reload the adapter. Keep dev as evaluation-only and avoid tuning against test outputs.

Inference and practical limitations

Inference uses original system/user messages, native chat formatting, greedy model.generate and 200 new tokens. Decode generated tokens with normal special-token omission and strip whitespace; no repairs or rules. Acceptance requires reloading the adapter and matching ten seeded test outputs. One seed and 200 dev rows limit conclusions; hidden test may be harder. Package/runtime drift is recorded in environment.json.